

CSE 4403: Algorithms

Lecture22

MaxFlow

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Maximum Flow Problem

Given a directed graph with edge capacities, a source s and a sink t , find the maximum total flow that can be routed from s to t without exceeding any edge's capacity.

□ Given

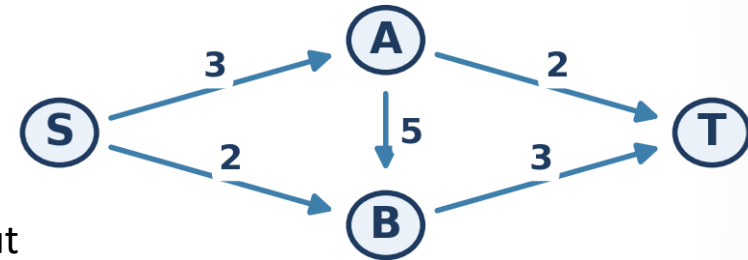
Directed graph $G=(V,E)$, capacity $c(u,v) \geq 0$ on each edge

□ Goal

Maximize the flow value: $\sum \text{flow}(s,u) = \sum \text{flow}(u,t) = \text{total out of } s = \text{total into } t$

□ Constraint

Capacity limit: $\text{flow}(u,v) \leq c(u,v)$ on every edge. Flow conservation: $\sum \text{flow}(v,u) = \sum \text{flow}(u,v)$ at every node except s and t



Real-world framing: maximizing traffic through roads, water through pipes, or data through a network.

What Are Residual Graphs & Augmenting Paths?

The residual graph tracks remaining capacity on each edge plus a reverse edge that lets flow already sent be “undone.” An augmenting path is any s-t path in this residual graph that still has spare capacity on every edge along it.

Residual Capacity

Forward edge (u,v) : $c(u,v) - \text{flow}(u,v)$.

Reverse edge (v,u) : $\text{flow}(u,v)$, letting the algorithm reroute a bad earlier choice.

Bottleneck & Termination

Each augmenting path can carry flow equal to its smallest residual capacity. When no s->t path remains in the residual graph, the flow is maximum.

Ford–Fulkerson repeats “find path, push flow” on the residual graph until no augmenting path is left.

Max Flow: Ford-Fulkerson Algorithm

Initialize flow

- 1 Build the residual graph with forward edge equal to capacity $c(u,v)$. This signifies the remaining capacity in forward direction. There is no reverse edge in the graph (i.e. their capacity is 0).

Find an augmenting path

- 2 Use BFS/DFS to find a path from s to t in the residual graph with available capacity > 0 on every edge.

Augment along the path

- 3 Push flow equal to the path's bottleneck (min residual capacity); update forward and reverse residual edges. Update in fw direction happens by deducting the flow. In the reverse direction, we add the current flow which signifies reverse capacity.

Repeat until stuck

- 4 Go back to Step 2. When no augmenting path remains, the current flow is maximum (max-flow min-cut).

Ford-Fulkerson: Pseudocode

```
FORD-FULKERSON(G, s, t):  
  for each edge (u, v) in E:  
    residual(u, v) = c(u, v)      // forward: starts at full capacity  
    residual(v, u) = 0            // reverse: starts at 0  
  
  while there is a path p from s to t with residual(u,v) > 0 on p:  
    bottleneck = min( residual(u, v) for (u, v) in p )  
  
    for each edge (u, v) in p:  
      residual(u, v) -= bottleneck // less room to push forward  
      residual(v, u) += bottleneck // more room to undo it later  
  
  return total flow out of s // sum of c(s,v) - residual(s,v)
```

Proof: Ford-Fulkerson Running Time

Claim: If all edge capacities are integers, Ford–Fulkerson terminates in $O(E \cdot f)$ time, where f is the value of the maximum flow.

Each augmenting path has bottleneck capacity $cf(p) = \min cf(u,v)$, where $(u,v) \in p$. Since every residual capacity is an integer ≥ 1 whenever the edge is usable, the bottleneck is always an integer ≥ 1 .

So every augmentation increases the total flow value by at least 1. Because the flow value can never exceed f (the max flow), the algorithm performs at most f augmentations before no augmenting path remains.

Each augmentation searches for a path with one BFS or DFS over the residual graph, which has $O(E)$ edges, so each search costs $O(E)$.

$$\begin{aligned} \text{Total time} &= (\text{number of augmentations}) \times (\text{cost per augmentation}) \\ &= O(f) \times O(E) = O(E \cdot f) \end{aligned}$$

What Is the Edmonds-Karp Algorithm?

Edmonds-Karp is Ford-Fulkerson with one rule added: always pick the augmenting path with the fewest edges, found by running a plain BFS on the residual graph each round. No other change to the algorithm is needed.

Why Shortest Path?

Plain Ford-Fulkerson can pick any augmenting path, including long, low-capacity ones. With irrational or large integer capacities this can loop almost forever.

Guaranteed Bound

Always taking the shortest path guarantees termination in $O(V \cdot E)$ augmentations in polynomial time, independent of edge capacities.

Each BFS costs $O(E)$, so total running time is $O(V \cdot E^2)$

Edmonds-Karp: Pseudocode

```
EDMONDS-KARP(G, s, t):  
    for each edge (u, v) in E:  
        residual(u, v) = c(u, v)  
        residual(v, u) = 0  
  
    while BFS(Gf, s, t) finds a path p:      // shortest path, fewest edges  
        bottleneck = min( residual(u, v) for (u, v) in p )  
  
        for each edge (u, v) in p:  
            residual(u, v) -= bottleneck  
            residual(v, u) += bottleneck  
  
    return total flow out of s  
  
BFS(Gf, s, t):  
    standard breadth-first search on the residual graph Gf  
    returns the s-t path using the fewest edges, or "none"
```


Proof: Edmonds-Karp Running Time (1/2)

Edmonds-Karp always augments along a shortest s-t path (fewest edges) in the residual graph, found by BFS.

Let $\delta_i(v)$ denote the BFS distance from s to v in the residual graph after i augmentations.

Lemma 1 (distances never decrease): *For every vertex v, $\delta_i(v)$ is non-decreasing as i increases.*

Sketch: Augmenting along a shortest path can only add reverse edges that point “backward” toward s. Using such a reverse edge later would require a shorter route than BFS already found earlier, which contradicts δ being a shortest-path distance at that earlier step.

This monotonicity is the key fact used to bound how many times any single edge can be the bottleneck of an augmenting path.

Proof: Edmonds-Karp Running Time (2/2)

An edge (u,v) is critical in an augmentation if it is the bottleneck, so it disappears from the residual graph afterward.

Lemma 2 (each edge is critical $O(V)$ times): When (u,v) is critical, $\delta(v) = \delta(u) + 1$. For (u,v) to become usable again, flow must later be pushed back along (v,u) , requiring $\delta'(u) = \delta'(v) + 1$ at that later point. So the second time,

$$\delta'(u) = \delta'(v) + 1,$$

Also by Lemma 1, $\delta'(v) \geq \delta(v)$

Adding 1 in both sides, $\delta'(v) + 1 \geq \delta(v) + 1$ or, $\delta'(u) \geq \delta(v) + 1$ or, $\delta'(u) \geq (\delta(u) + 1) + 1$ or, $\delta'(u) \geq \delta(u) + 2$

So between any two consecutive times (u,v) is critical, $\delta(u)$ has gone up by at least 2. Since $\delta(v)$ never decreases (Lemma 1), this jump of ≥ 2 keeps happening every time (u,v) becomes critical again.

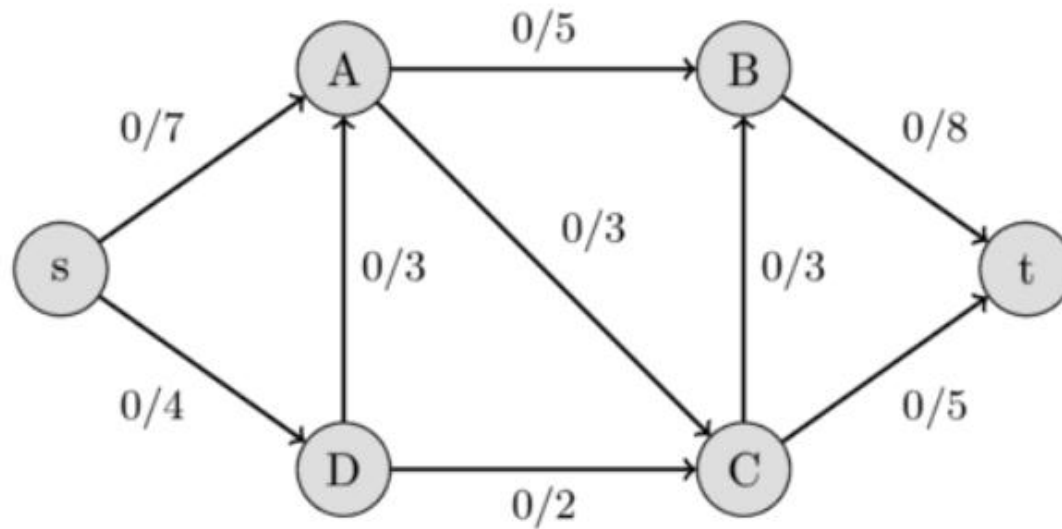
$\delta(u)$ can only take the V possible values $0, 1, \dots, V-1$. Starting from $\delta(u) \geq 0$ and rising by ≥ 2 each time, it can hit at most about $V/2$ distinct values before running out. So (u,v) can be critical at most $O(V)$ times.

There are $O(E)$ edges, so total critical-edge events and hence number of augmentations is $O(V \cdot E)$. Each augmentation costs $O(E)$ for its BFS.

$$\begin{aligned} \text{Total time} &= (\text{number of augmentations}) \times (\text{cost per augmentation}) \\ &= O(V \cdot E) \times O(E) = O(V \cdot E^2) \end{aligned}$$

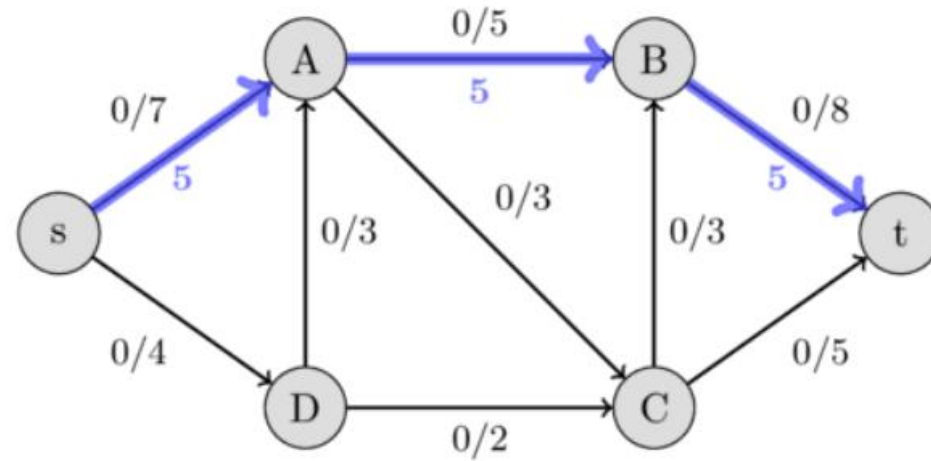
Worked Example: The Network

This is the reference example from cp-algorithms.com's Ford-Fulkerson / Edmonds-Karp article : six nodes, nine directed edges. Each edge is labeled flow/capacity, so it starts at 0/capacity everywhere.



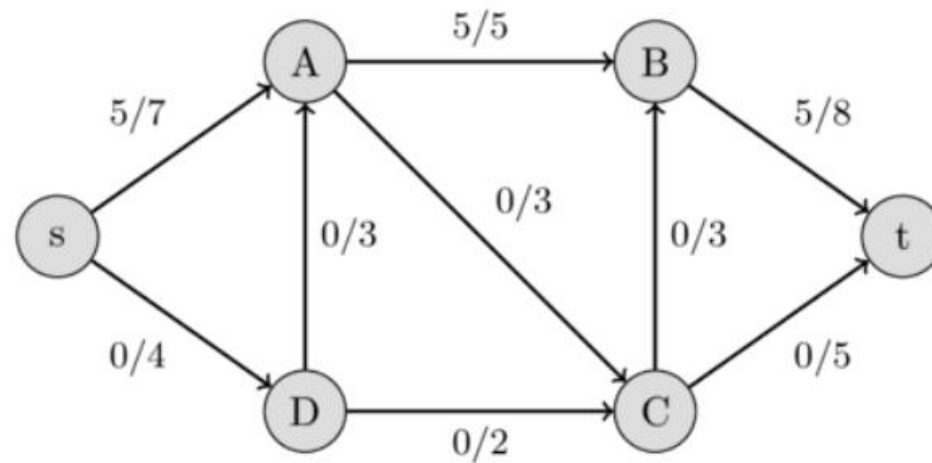
Worked Example

We can find the path $s - A - B - t$ with the residual capacities 7, 5, and 8. Their minimum is 5, therefore we can increase the flow along this path by 5. This gives a flow of 5 for the network.



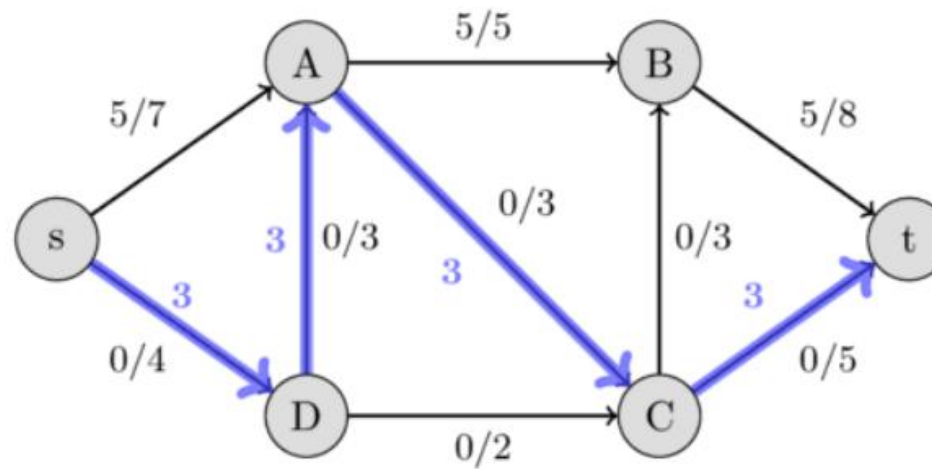
Worked Example

We can find the path $s - A - B - t$ with the residual capacities 7, 5, and 8. Their minimum is 5, therefore we can increase the flow along this path by 5. This gives a flow of 5 for the network.



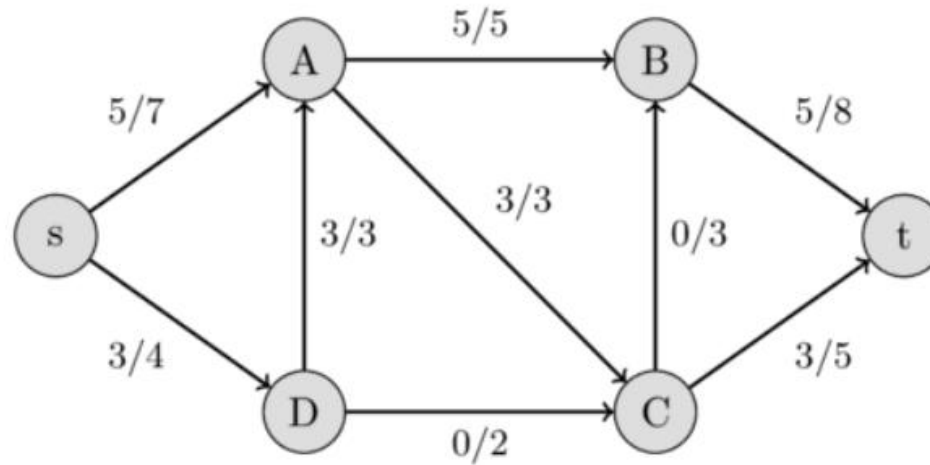
Worked Example

Again we look for an augmenting path, this time we find $s - D - A - C - t$ with the residual capacities 4, 3, 3, and 5. Therefore we can increase the flow by 3 and we get a flow of 8 for the network.



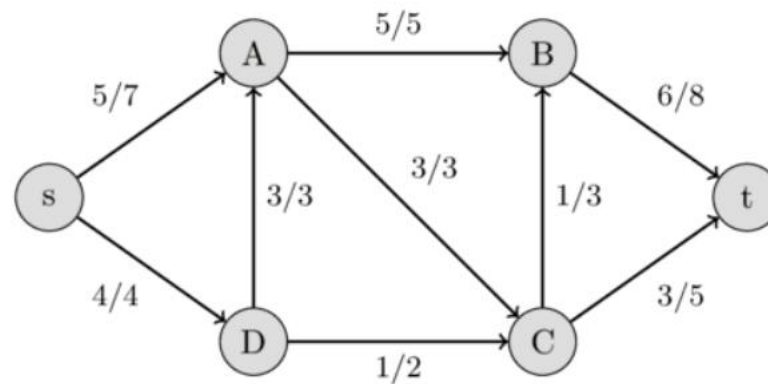
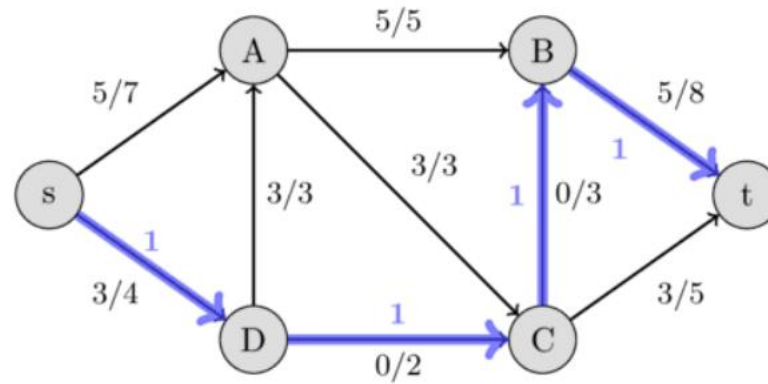
Worked Example

Again we look for an augmenting path, this time we find $s - D - A - C - t$ with the residual capacities 4, 3, 3, and 5. Therefore we can increase the flow by 3 and we get a flow of 8 for the network.



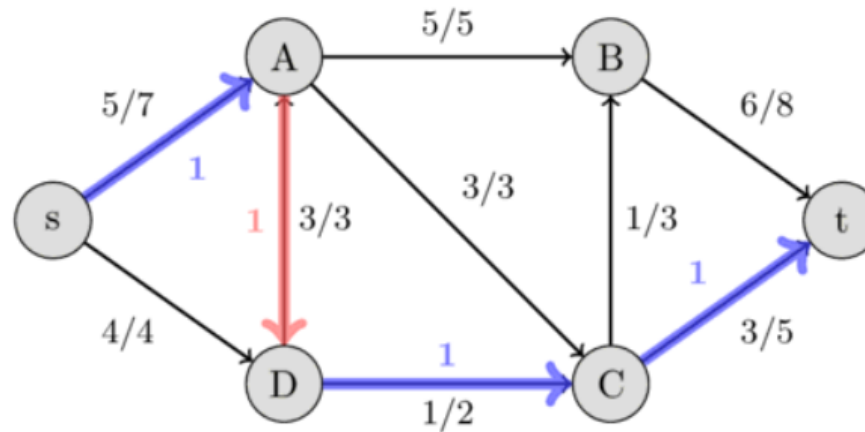
Worked Example

This time we find the path $s - D - C - B - t$ with the residual capacities 1, 2, 3, and 3, and hence, we increase the flow by 1.

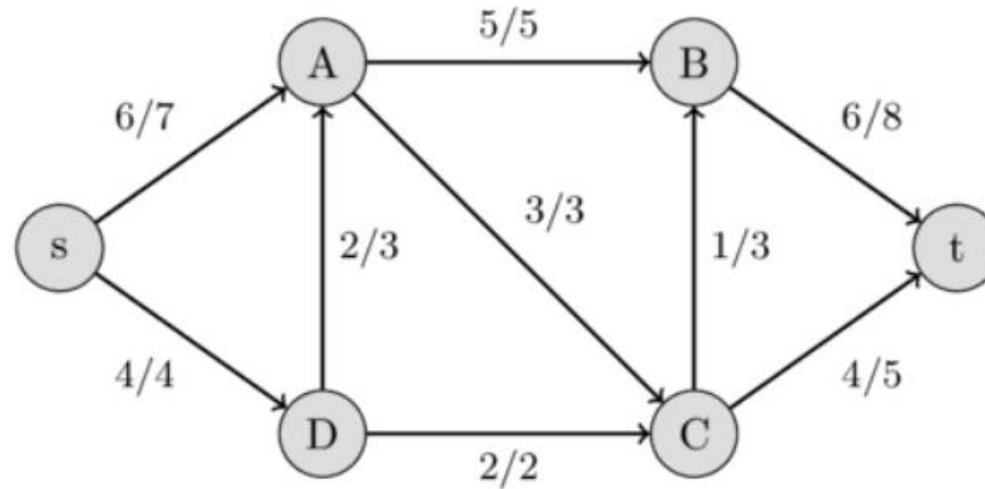


Worked Example

This time we find the augmenting path $s - A - D - C - t$ with the residual capacities 2, 3, 1, and 2. We can increase the flow by 1. But this path is very interesting. It includes the reversed edge (A, D) . In the original flow network, we are not allowed to send any flow from A to D . But because we already have a flow of 3 from D to A , this is possible. The intuition of it is the following: Instead of sending a flow of 3 from D to A , we only send 2 and compensate this by sending an additional flow of 1 from s to A , which allows us to send an additional flow of 1 along the path $D - C - t$.



Worked Example: The Result



Now, it is impossible to find an augmenting path between s and t , therefore this flow of 10 is the maximal possible. We have found the maximal flow.

Worked Example: Tracing Ford–Fulkerson

Network from the previous slide (cp-algorithms.com)

Iter	Augmenting Path	Bottleneck	Flow So Far
1	$s \rightarrow A \rightarrow B \rightarrow t$	5	5
2	$s \rightarrow D \rightarrow A \rightarrow C \rightarrow t$	3	8
3	$s \rightarrow D \rightarrow C \rightarrow B \rightarrow t$	1	9
4	$s \rightarrow A \rightarrow D \rightarrow C \rightarrow t$ (uses reverse edge $D \rightarrow A$)	1	10
5	None remain in residual graph	—	10 (max)

Maximum Flow = 10

Complexity

Finding one augmenting path

$O(E)$ via a single BFS or DFS over the residual graph, with V vertices and E edges.

Number of augmentations

Plain Ford-Fulkerson can need up to f iterations, where f is the max flow value, so it is slow if capacities are large.

Edmonds-Karp (always augment along the shortest path via BFS) bounds this to $O(V \cdot E)$ iterations regardless of capacity size.

$$O(E)$$

BFS/DFS per augmenting path

$$O(E \cdot f)$$

Ford-Fulkerson (f = max flow value)

$$O(V \cdot E^2)$$

Edmonds-Karp bound (BFS-based)